

7. Модель ассоциативной памяти на основе сетей Хопфилда

Главы книги

Введение в сети Хопфилда см. в [главе 17, раздел 2](#).

Классы Python

Сети Хопфилда можно анализировать математически. В этом упражнении на Python мы сосредоточимся на визуализации и моделировании, чтобы лучше понять динамику сетей Хопфилда.

Мы предоставляем несколько функций для простого создания шаблонов, их сохранения в сети и визуализации сетевой динамики. Ознакомьтесь с модулями `hopfield_network.network`, `hopfield_network.pattern_tools` и `hopfield_network.plot_tools`, чтобы узнать о предоставляемых нами базовых элементах.

! Примечание

Если вы создаете экземпляр нового объекта класса `network.HopfieldNetwork`, его динамика по умолчанию является **детерминированной** и **синхронной**. То есть все состояния обновляются одновременно с помощью функции `sign`. Мы используем эту динамику во всех упражнениях, описанных ниже.

7.1. Начало работы:

Запустите следующий код. Ознакомьтесь с комментариями и документацией. Узоры и перевернутые пиксели выбираются случайным образом. Поэтому результат меняется при каждом запуске этого кода. Запустите его несколько раз и измените некоторые параметры, например `nr_patterns` и `nr_of_flips`.

```

%matplotlib inline
from neurodynex3.hopfield_network import network, pattern_tools, plot_tools

pattern_size = 5

# создаем экземпляр класса HopfieldNetwork
hopfield_net = network.HopfieldNetwork(nr_neurons= pattern_size**2)
# создаем экземпляр фабрики паттернов
factory = pattern_tools.PatternFactory(pattern_size, pattern_size)
# создайте шаблон шахматной доски и добавьте его в список шаблонов
checkboard = factory.create_checkboard()
pattern_list = [шахматная доска]

# добавьте случайные шаблоны в список
pattern_list.расширить(factory.create_random_pattern_list(nr_patterns=3,
on_probability=0.5))
plot_tools.plot_pattern_list(список шаблонов)
# насколько похожи случайные шаблоны и шахматная доска? Проверьте перекрытие
overlap_matrix = pattern_tools.compute_overlap_matrix(pattern_list)
plot_tools.plot_overlap_matrix(overlap_matrix)

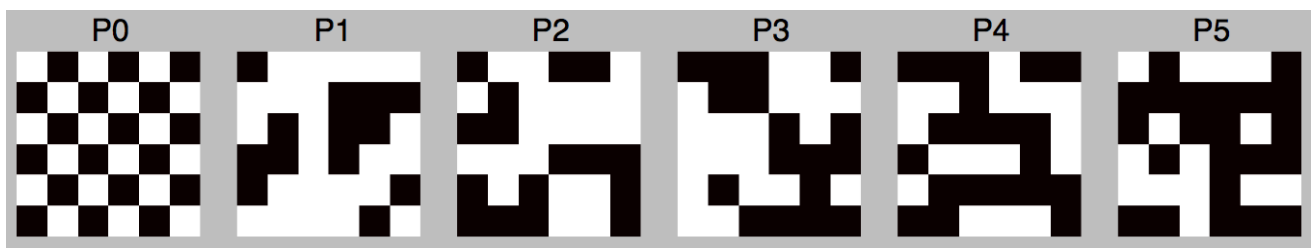
# дайте нейросети Хопфилда «изучить» паттерны. Примечание: они не сохраняются
# явно, обновляются только весовые коэффициенты сети !
hopfield_net.store_patterns(pattern_list)

# создаем зашумленную версию паттерна и используем ее для инициализации сети
noisy_init_state = pattern_tools.flip_n(checkboard, nr_of_flips=4)
hopfield_net.set_state_from_pattern(noisy_init_state)

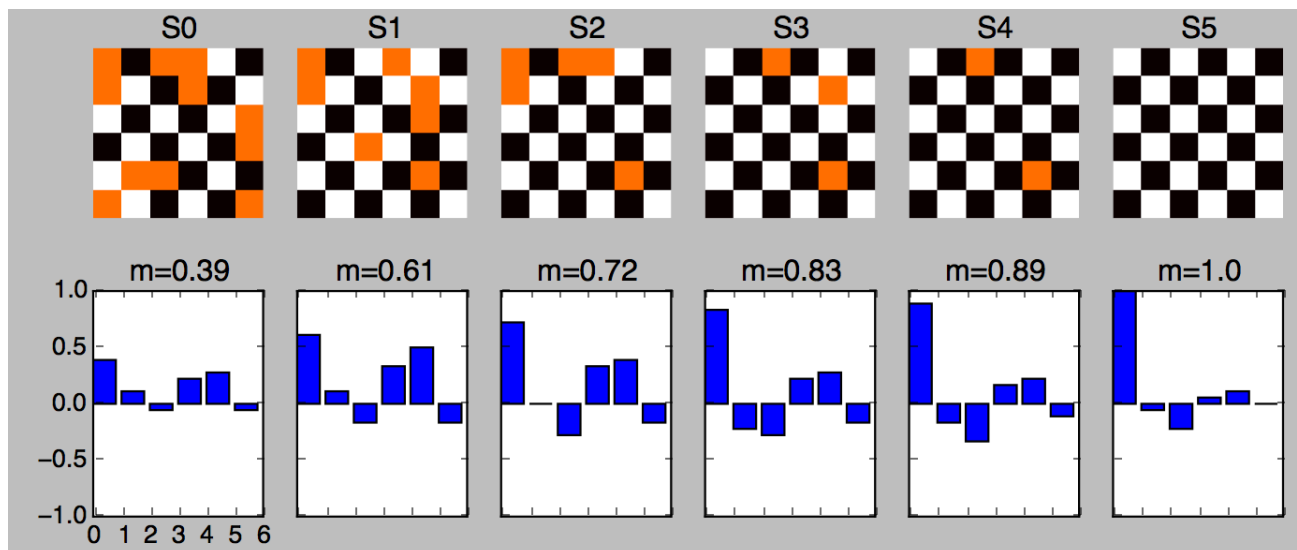
# Пусть динамика сети развивается из этого исходного состояния.
states = hopfield_net.run_with_monitoring(nr_steps=4)

# каждое состояние сети представлено вектором. преобразуем его в ту же форму, которая
использовалась для создания паттернов.
states_as_patterns = factory.reshape_patterns(states)
# строим график состояний сети
plot_tools.plot_state_sequence_and_overlap(states_as_patterns, pattern_list,
reference_idx=0, supitle="Динамика сети")

```



В сети Хопфилда хранится шесть шаблонов.



Сеть инициализируется (очень) зашумленным паттерном $S(t=0)$. Затем динамика восстанавливает исходный паттерн P_0 за 5 итераций.

! Примечание

Состояние сети — это вектор N нейронов. Для визуализации мы используем двумерные паттерны, которые представляют собой `numpy.ndarray` объекты размером = (длина, ширина). Чтобы сохранить такие паттерны, инициализируйте сеть `N = length * width` нейронами.

7.2. Введение: сети Хопфилда

В этом упражнении используется модель, в которой нейроны представлены пикселями и принимают значения -1 (выключено) или $+1$ (включено). Сеть может хранить определенное количество пиксельных паттернов, что и будет изучаться в этом упражнении. На этапе извлечения данных сеть запускается с некоторой начальной конфигурацией, и динамика сети стремится к сохраненному паттерну (аттрактору), наиболее близкому к начальной конфигурации.

Динамика соответствует уравнению:

$$S_i(t+1) = \text{sgn} \left(\sum_j w_{ij} S_j(t) \right)$$

В модели Хопфилда каждый нейрон связан со всеми остальными нейронами (полная связность). Матрица связей имеет вид

$$w_{ij} = \frac{1}{N} \sum_{\mu} p_i^{\mu} p_j^{\mu}$$

где N — количество нейронов, p_i^{μ} — значение нейрона i в шаблоне номер μ и сумма

работает, когда паттерны, которые нужно сохранить, являются случайными и с равной вероятностью могут быть как включенными (+1), так и выключенными (-1). В больших сетях ($N \rightarrow \infty$) количество случайных последовательностей, которые можно сохранить, составляет примерно $0.14N$.

7.3. Exercise: N=4x4 Hopfield-network

We study how a network stores and retrieve patterns. Using a small network of only 16 neurons allows us to have a close look at the network weights and dynamics.

7.3.1. Question: Storing a single pattern

Modify the Python code given above to implement this exercise:

1. Create a network with $N = 16$ neurons.
2. Create a single 4 by 4 checkerboard pattern.
3. Store the checkerboard in the network.
4. Set the initial state of the network to a noisy version of the checkerboard (`nr_flipped_pixels = 5`).
5. Let the network dynamics evolve for 4 iterations.
6. Plot the sequence of network states along with the overlap of network state with the checkerboard.

Now test whether the network can still retrieve the pattern if we increase the number of flipped pixels. What happens at `nr_flipped_pixels = 8`, what if `nr_flipped_pixels > 8`?

7.3.2. Question: the weights matrix

The patterns a Hopfield network learns are not stored explicitly. Instead, the network learns by adjusting the weights to the pattern set it is presented during learning. Let's visualize this.

1. Create a new 4x4 network. Do not yet store any pattern.
2. What is the size of the network matrix?
3. Visualize the weight matrix using the function `plot_tools.plot_nework_weights()`. It takes the network as a parameter.
4. Create a checkerboard, store it in the network.
5. Plot the weights matrix. What weight values do occur?
6. Create a new 4x4 network.
7. Create an L-shaped pattern (look at `pattern_tools.PatternFactory`), store it in the network.
8. Plot the weights matrix. What weight values do occur?
9. Create a new 4x4 network.
10. Create a checkerboard and an L-shaped pattern. Store **both** pattern

! Note

The mapping of the 2-dimensional patterns onto the one-dimensional list of network neurons is internal to the implementation of the network. You cannot know which pixel (x,y) in the pattern corresponds to which network neuron i.

7.3.3. Question (optional): Weights Distribution

It's interesting to look at the weights distribution in the three previous cases. You can easily plot a histogram by adding the following two lines to your script. It assumes you have stored your network in the variable `hopfield_net`.

```
plt.figure()
plt.hist(hopfield_net.weights.flatten())
```

7.4. Exercise: Capacity of an N=100 Hopfield-network

Larger networks can store more patterns. There is a theoretical limit: the capacity of the Hopfield network. Read [chapter "17.2.4 Memory capacity"](#) to learn how memory retrieval, pattern completion and the network capacity are related.

7.4.1. Question:

A Hopfield network implements so called **associative** or **content-adressable** memory. Explain what this means.

7.4.2. Question:

Using the value C_{store} given in the book, how many patterns can you store in a N=10x10 network? Use this number K in the next question:

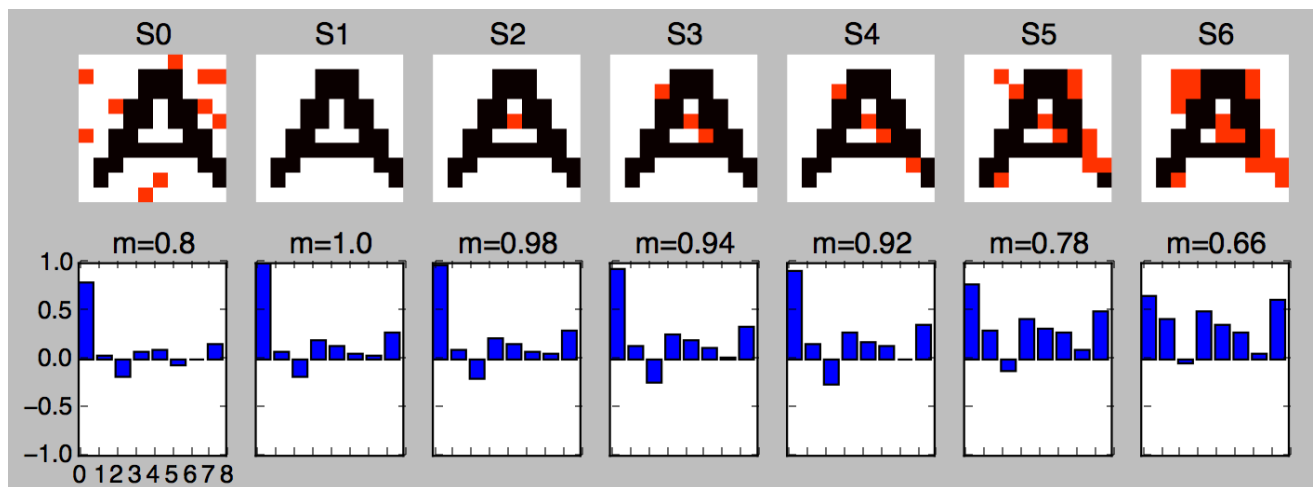
7.4.3. Question:

Create an N=10x10 network and store a checkerboard pattern together with $(K - 1)$ **random patterns**. Then initialize the network with the **unchanged** checkerboard pattern. Let the network evolve for five iterations.

Rerun your script a few times. What do you observe?

7.5. Exercise: Non-random patterns

In the previous exercises we used random patterns. Now we use a list of structured patterns:



Eight letters (including 'A') are stored in a Hopfield network. The letter 'A' is not recovered.

7.5.1. Question:

Run the following code. Read the inline comments and look up the doc of functions you do not know.

```

%matplotlib inline
import matplotlib.pyplot as plt
from neurodynex3.hopfield_network import network, pattern_tools, plot_tools
import numpy

# the letters we want to store in the hopfield network
letter_list = ['A', 'B', 'C', 'S', 'X', 'Y', 'Z']

# set a seed to reproduce the same noise in the next run
# numpy.random.seed(123)

abc_dictionary = pattern_tools.load_alphabet()
print("the alphabet is stored in an object of type: {}".format(type(abc_dictionary)))
# access the first element and get it's size (they are all of same size)
pattern_shape = abc_dictionary['A'].shape
print("letters are patterns of size: {}. Create a network of corresponding
size".format(pattern_shape))
# create an instance of the class HopfieldNetwork
hopfield_net = network.HopfieldNetwork(nr_neurons= pattern_shape[0]*pattern_shape[1])

# create a list using Pythons List Comprehension syntax:
pattern_list = [abc_dictionary[key] for key in letter_list ]
plot_tools.plot_pattern_list(pattern_list)

# store the patterns
hopfield_net.store_patterns(pattern_list)

# # create a noisy version of a pattern and use that to initialize the network
noisy_init_state = pattern_tools.get_noisy_copy(abc_dictionary['A'], noise_level=0.2)
hopfield_net.set_state_from_pattern(noisy_init_state)

# from this initial state, let the network dynamics evolve.
states = hopfield_net.run_with_monitoring(nr_steps=4)

# each network state is a vector. reshape it to the same shape used to create the
patterns.
states_as_patterns = pattern_tools.reshape_patterns(states, pattern_list[0].shape)

# plot the states of the network
plot_tools.plot_state_sequence_and_overlap(
    states_as_patterns, pattern_list, reference_idx=0, suptitle="Network dynamics")

```

7.5.2. Question:

Add the letter 'R' to the letter list and store it in the network. Is the pattern 'A' still a fixed point? Does the overlap between the network state and the reference pattern 'A' always decrease?

7.5.3. Question:

Make a guess of how many letters the network can store. Then create a (small) set of letters. Check if **all** letters of your list are fixed points under the network dynamics. Explain the discrepancy between the network capacity C (computed above) and your observation.

7.6. Exercise: Bonus

The implementation of the Hopfield Network in `hopfield_network.network` offers a possibility to provide a custom update function

`HopfieldNetwork.set_dynamics_to_user_function()`. Have a look at the source code of

`HopfieldNetwork.set_dynamics_sign_sync()` to learn how the update dynamics are implemented. Then try to implement your own function. For example, you could implement an asynchronous update with stochastic neurons.

